

Deep Learning Practical 4: Model answers

Tensor operations and convolutional architectures

2 Tensor manipulation with Einstein operations

Construct a two-layer MLP using **Rearrange**, with five hidden units and one output. Confirm that `net(coffee_batch).size()` is `(10, 1)`.

Flattening preserves the batch axis and combines each photograph's three colour channels and 400×600 pixels into 720,000 input features:

```
from einops.layers.torch import Rearrange
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            Rearrange("b c h w -> b (c h w)"),
            nn.Linear(3*400*600, 5), nn.ReLU(), nn.Linear(5, 1))
    def forward(self, x):
        return self.layers(x)
net = MLP()
assert net(coffee_batch).shape == (10, 1)
```

This gives one output per image and contains $720000 \times 5 + 5 + 5 \times 1 + 1 = 3,600,011$ parameters.

Run the mosaic rearrangement and explain what is happening.

Splitting the ten-image batch into two groups of five arranges the copies as a grid with two rows and five columns. Combining each group index with its corresponding spatial axis gives an image of shape $(800, 3000, 3)$. Unlike `reduce`, which aggregates values, `rearrange` changes only their layout. The earlier `repeat` operation created the ten copies.

3 Convolutional neural networks

1. Design a five-layer convolutional encoder taking $B \times 3 \times 32 \times 32$ to $B \times 1024 \times 1 \times 1$, without pooling.

Using kernel size 4, stride 2 and padding 1 halves both spatial dimensions at each layer. One suitable sequence of channel counts and spatial sizes is

$$3 \times 32^2 \rightarrow 32 \times 16^2 \rightarrow 64 \times 8^2 \rightarrow 128 \times 4^2 \rightarrow 256 \times 2^2 \rightarrow 1024 \times 1^2.$$

ReLU between the convolutions makes the encoder a nonlinear function.

2. Make an Encoder class using nn.Sequential and confirm its output shape.

```
class Encoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Conv2d(3, 32, 4, 2, 1), nn.ReLU(),
            nn.Conv2d(32, 64, 4, 2, 1), nn.ReLU(),
            nn.Conv2d(64, 128, 4, 2, 1), nn.ReLU(),
            nn.Conv2d(128, 256, 4, 2, 1), nn.ReLU(),
            nn.Conv2d(256, 1024, 4, 2, 1))
    def forward(self, x):
        return self.layers(x)
x0 = torch.zeros(10, 3, 32, 32)
assert Encoder()(x0).shape == (10, 1024, 1, 1)
```

3. How many parameters does each layer use?

A convolution with a bias contains $C_{\text{out}}(C_{\text{in}}k_hk_w + 1)$ parameters. Applying this to the five layers gives:

Channels	Parameters
$3 \rightarrow 32$	1,568
$32 \rightarrow 64$	32,832
$64 \rightarrow 128$	131,200
$128 \rightarrow 256$	524,544
$256 \rightarrow 1024$	4,195,328
Total	4,885,472

The final layer dominates because it connects 256 input channels to 1,024 output channels through a 4×4 kernel.

4. How could the encoder use fewer parameters while still producing 1,024 latent values?

Reducing the intermediate channel counts to 3, 16, 32, 64, 64, 1024, while keeping the same kernels, strides and padding, gives 1,157,040 parameters. It still ends at $1024 \times 1 \times 1$, although fewer intermediate features may reduce its ability to represent the data.

5. How would you adjust the architecture for inputs of size $B \times 3 \times 32 \times 64$?

Keep the first four layers, which now end at $B \times 256 \times 2 \times 4$. Change the last convolution to `nn.Conv2d(256, 1024, (4,6), (2,2), (1,1))`. The output-size formula gives $\lfloor (2+2-4)/2 \rfloor + 1 = 1$ in height and $\lfloor (4+2-6)/2 \rfloor + 1 = 1$ in width, so the five-layer encoder still produces 1,024 values per image.

6. Design a five-layer convolutional decoder taking $B \times 1024 \times 1 \times 1$ back to $B \times 3 \times 32 \times 32$.

Transposed convolutions with the same 4,2,1 pattern double the spatial size. Reversing the channel sequence gives:

```
class Decoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.ConvTranspose2d(1024, 256, 4, 2, 1), nn.ReLU(),
            nn.ConvTranspose2d(256, 128, 4, 2, 1), nn.ReLU(),
            nn.ConvTranspose2d(128, 64, 4, 2, 1), nn.ReLU(),
            nn.ConvTranspose2d(64, 32, 4, 2, 1), nn.ReLU(),
            nn.ConvTranspose2d(32, 3, 4, 2, 1))
    def forward(self, x):
        return self.layers(x)
assert Decoder()(Encoder()(x0)).shape == x0.shape
```

Its spatial sizes are 1, 2, 4, 8, 16, 32, and its parameter count is 4,884,451. Matching the input shape does not make this an inverse of the encoder: reconstruction requires training the two networks together.

5 Block design

Add ResidualBlock layers to the encoder and decoder. Check their shapes and parameter counts.

Using the batch-normalised block from Section 4, insert a block after each convolution except the final one:

```
def add_blocks(network):
    layers = []
    for layer in network.layers:
        layers.append(layer)
        if isinstance(layer, (nn.Conv2d, nn.ConvTranspose2d)):
            if layer.out_channels in (32, 64, 128, 256):
                layers.append(ResidualBlock(layer.out_channels))
    network.layers = nn.Sequential(*layers)
    return network
```

Each block preserves its input shape. At width n , its two convolutions and two batch-normalisation layers add $18n^2 + 6n$ trainable parameters. The four widths therefore add 1,569,600 parameters to each network, giving 6,455,072 in the encoder and 6,454,051 in the decoder. Batch-normalisation running statistics are stored buffers and are excluded from these parameter counts.

Create an Autoencoder class and confirm that it returns a tensor with the same shape as x_0 .

```
class Autoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = add_blocks(Encoder())
        self.decoder = add_blocks(Decoder())
    def forward(self, x):
        return self.decoder(self.encoder(x))
net = Autoencoder()
assert net.encoder(x0).shape == (10, 1024, 1, 1)
assert net(x0).shape == (10, 3, 32, 32)
assert sum(p.numel() for p in net.parameters()) == 12909123
```

6 Improve the CIFAR-10 colouriser

1. Check the shapes at each residual addition. Why does adding the input change the Jacobian from J_f to $I + J_f$ before the final activation?

Within each residual block, both 3, 1, 1 convolutions preserve the number of channels and the spatial dimensions, so $f(x)$ and x have equal shapes. Differentiating $x + f(x)$ gives $I + J_f(x)$ because the derivative of x with respect to itself is I . The final ReLU can still set some components of this combined derivative to zero.

The two encoder stages produce $(B, 16, 16, 16)$ and $(B, 32, 8, 8)$ respectively. Upsampling the second output returns $(B, 16, 16, 16)$, matching the first-stage features for the optional feature skip.

2. Replace one stage by a stage with no residual blocks. Compare validation MSE, parameter count and images after 6,000 updates.

Removing the second stage's two residual blocks can be done before creating the optimiser:

```
model.encoder[1].layers = model.encoder[1].layers[:2]
```

These blocks omit batch normalisation, so each has $18n^2 + 2n$ parameters. At width 32, removing both saves 36,992 parameters and reduces the network from 64,467 to 27,475. With 6,000 CPU updates, weight seed 3801, batch seed 3802, batch size 64 and learning rate 0.001, validation MSE rises from 0.005099 to 0.005139. The same eight images retain sharp grey outlines and broadly similar colours, so the parameter reduction has a small effect in this run. These CPU values are a fresh comparison and differ slightly from the recorded GPU example.

3. Compare `feature_skip=False` with `feature_skip=True`.

Enabling the feature skip adds the encoder's 16×16 features to the upsampled bottleneck output without introducing any parameters. This gives the decoder direct access to features that would otherwise need to survive another downsampling stage. Under the same settings, validation MSE falls from 0.005099 to 0.005031. Changes in the eight images are modest, with some shifts in colour rather than a dramatic gain in sharpness, since the direct grey path already carries the outlines. This small single-run advantage should not be treated as a guaranteed improvement across seeds.

4. Remove the direct greyscale path. Does the model spend more capacity reproducing luminance? Compare the same eight examples.

Returning `self.decoder(up)` removes the grey addition. Now the model must reconstruct brightness and edges as well as colour, and its zero-initialised output layer starts with a black image instead of the grey baseline. In the matched CPU run, RGB MSE increases from 0.005099 to 0.006297 and luminance MSE from 3.49×10^{-6} to 9.93×10^{-4} . Edges in the sail, vehicle and animal faces are visibly less crisp without the grey path, consistent with the increased luminance error. The parameter count stays at 64,467. Greater luminance error shows that brightness is reproduced less accurately, although it does not identify a particular number of parameters devoted exclusively to luminance.

5. Compare one wider stage with two narrower residual blocks at a similar parameter count, keeping the data, seed, batch and updates fixed.

Replace the two width-16 blocks in the first stage by one residual branch with convolutions $16 \rightarrow 32 \rightarrow 16$, a ReLU between them and a ReLU after the addition. This branch has 9,264 parameters, compared with 9,280 in the two original blocks, giving total counts of 64,451 and 64,467 respectively. It trades successive nonlinear transformations for more features within one transformation. After the same 6,000 updates, the wider branch gives validation MSE 0.005077, compared with 0.005099 for the two narrower blocks. The reconstructions look similar, so this is a small improvement for this run rather than a strong case that width generally matters more than depth.

6. Use `reduce` to examine per-channel error and `rearrange` to show input, output and target rows in one image.

```
with torch.no_grad():
    target = validation[:8].to(device)
    mono = grey(target)
    output = model(mono).clamp(0, 1)
    error = (output-target).square()
    print(reduce(error, 'b c h w -> c', 'mean'))
    rows = torch.stack([mono.expand(-1, 3, -1, -1), output, target])
    grid = rearrange(rows, 'r b c h w -> (r h) (b w) c')
plt.imshow(grid.cpu())
plt.axis('off')
```

The reduction returns three errors in RGB order. Stacking introduces a row axis, which the rearrangement combines with image height to place input, prediction and target above one another.

Does more training improve the colours?

Compare 1,500, 6,000 and 15,000 updates, inspecting the same photographs and plotting training and validation error.

In the supplied comparison at learning rate 0.002, validation MSE is 0.005264, 0.005164 and 0.005218 respectively. Although training error continues to fall, validation error is lowest at 6,000 updates. The later training therefore fits the training examples more closely without improving the average prediction on unseen photographs. Muted colours can also persist because squared error favours averaging when the input leaves colour uncertain.

7 Apply the convolutional network to coffee

Why can the convolutional network accept this resolution, while the coffee MLP needs a fixed input size? Diagnose dimensions that are not multiples of four. Does grey determine coffee colour uniquely, and how would you assess accuracy?

A convolution applies the same small kernel at every location, so its weights can process different spatial sizes. The MLP instead has one weight for every input-feature-hidden-unit pair and therefore requires the same flattened input size as its first linear layer.

Two stride-two encoder stages followed by two doublings produce a correction of size $4\lfloor H/4\rfloor \times 4\lfloor W/4\rfloor$. For example, a 129×193 input gives a 128×192 correction, which cannot be added to the original grey input. Padding the input to multiples of four and cropping the final prediction resolves this mismatch. The feature skip also requires matching intermediate dimensions.

Many RGB colours share a grey value, so grey alone cannot determine the coffee's original colour uniquely. Compare the prediction with the original RGB photograph resized in exactly the same way, reporting RGB error alongside visual differences in the coffee, cup and background. A plausible brown drink may still differ from the photographed colour, and the model was trained on small CIFAR-10 images rather than coffee photographs at this resolution.